



2.14. Основы алгоритмов, часть 1

Всем привет!

На связи шпаргалка урока 2.14. Основы алгоритмов, часть 1.

Время прочтения: 14 минут.

[Вводная часть](#)

[Оцениваем сложность алгоритмов](#)

[О-нотация \(«О» большое\)](#)

[Структуры данных](#)

[Stack](#)

[Queue/Deque](#)

[Tree](#)

[Graph](#)

Вводная часть

Алгоритмы встречаются повсеместно. Даже когда вам нужно утром выйти из дома, вы осуществляете некий (зачастую стандартный) алгоритм:

1. Встаете с кровати.
2. Умываетесь.
3. Завтракаете.
4. Мойте посуду.
5. Одеваетесь.
6. Открываете дверь.
7. Закрываете дверь.
8. Выходите на улицу.



Эта последовательность тоже является алгоритмом.

Алгоритм является последовательным набором инструкций, которые требуется выполнить для решения определенной задачи.



Программирование целиком и полностью состоит из написания алгоритмов.

Нужно прочитать элементы массива — пишем **алгоритм**.

Нужно посчитать что-то и напечатать в консоль — тоже **алгоритм**.

Любой метод содержит в себе набор инструкций в определенном порядке.

Безусловно, с целью оптимизации Java может менять порядок выполнения ваших инструкций, но только таким образом, чтобы финальный результат оставался прежним.

Оцениваем сложность алгоритмов

О-нотация («О» большое)



С помощью этого параметра вычисляется сложность алгоритма.

Сложность обычно вычисляется в двух метриках: **время выполнения** и **память**.

Так как абсолютное значение времени выполнения подсчитать невозможно, ведь она может отличаться в зависимости от языка программирования, типа данных, процессора и еще множества различных факторов, нас интересует асимптотическая сложность выполнения алгоритма, т. е. сложность относительно количества обрабатываемых данных.

Именно для этого используется О-нотация или «О» большое.

Рассмотрим несколько популярных алгоритмов и попробуем посчитать их сложность.

Возьмем для примера задачу из ДЗ по массивам, где требовалось найти максимальный элемент.

▼ Текст задания

Нужно найти минимальную и максимальную трату за день.

Нужно написать программу, которая решит эту задачу, и вывести в консоль результат в формате: «Минимальная сумма трат за день составила ... рублей. Максимальная сумма трат за день составила ... рублей».

У нас имеется неотсортированный массив на некоторое количество числовых элементов. Его реальный размер нас не интересует, так как алгоритм не становится сложнее или легче от этого. Задача, безусловно, будет выполнена быстрее на малом количестве данных, чем на большом, но сложность алгоритма от этого не меняется.

Допустим, что в массиве содержится N элементов.

Для решения задачи с поиском максимального значения нам требуется пройти по всем элементам и сравнить их с текущим максимумом. Это значит, что алгоритм должен проделать столько шагов, сколько имеется элементов, т. е. N шагов.

В этом случае сложность будет **линейная** или **$O(n)$** .

Многие алгоритмы вроде поиска элемента в массиве могут сработать за один шаг (если необходимый элемент окажется в самой первой ячейке), но алгоритмы принято оценивать по худшему сценарию, т. е. по сценарию, когда элемента вообще нет в массиве.

В этом случае ваш алгоритм пройдет по всем элементам и вернет результат, что элемент не был найден.

Рассмотрим еще один пример, а именно поиск по индексу (в случае массива) или по ключу (в случае map).



Как вы помните, обращение по индексу всегда происходит сразу, т. е. за один шаг.

Никакого прохода по элементам в данном случае нет.

В этой ситуации сложность будет **константная** или **$O(1)$** .

Следующим примером будет **бинарный поиск** в отсортированном массиве.

Этот вид поиска проверяет центральный элемент и, если он больше или меньше необходимого, половина всех значений отбрасывается.

Например, если массив состоит из чисел 1, 2, 3, 4, 5 и вам нужно найти индекс 2, то:

1. Алгоритм возьмет 3 (центральный элемент) и сравнит его с 2 (необходимый элемент).
2. Т. к. 3 больше 2, алгоритм откинет всю правую часть и будет искать уже в левой части.
3. Следующее сравнение нового центрального элемента с двойкой откинет левую часть, которая состоит из единицы.
4. Центральным элементом станет двойка, и при сравнении с необходимым элементом будет выявлено равенство, а индекс элемента будет возвращен.

Как видите, вместо 5 сравнений (в худшем случае) нам потребовалось сделать 3.

И чем больше элементов в массиве, тем быстрее будет работать бинарный поиск.

Например, для поиска по 100 элементам бинарному поиску потребуется максимум 7 сравнений, для 1000 потребуется 10, а для миллиона — 20.

Этот алгоритм имеет **логарифмическую** сложность и записывается как **$O(\log_2 N)$** .

Последним примером возьмем самую простую сортировку — **пузырьковую**.

В этой сортировке два цикла.

1. Первый цикл отвечает за количество проходов по массиву, а второй — за сравнение текущего элемента со следующим. Если текущий элемент больше следующего, они меняются местами.
В результате первого цикла самый большой элемент должен встать на последнюю позицию.
2. На следующей итерации «борьба» идет уже за предпоследнюю позицию, так как элемент для последней позиции был проставлен в первой итерации.

И так далее.

У данной сортировки много оптимизаций, но в базовой реализации нам приходится в n итераций сравнивать n элементов друг с другом, проходя по каждому из них.

Сложность получается **квадратичной**, т. е. $n * n$. Записывается как **$O(n^2)$** .

Структуры данных

Мы с вами уже познакомились с самыми популярными реализациями различных структур данных.

Массивы и цепочки узлов используются повсеместно, однако бывают ситуации, когда нужна некоторая очень узконаправленная структура данных.

Ниже мы разберем возможности структур данных и области применения.

Stack

С этой структурой вы уже знакомы из урока 2.1, где в шпаргалке разбирался стек вызовов.

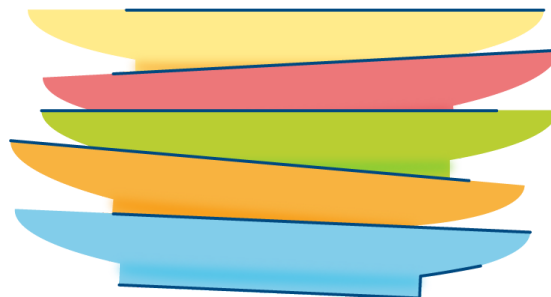


Это структура, обычно построенная на базе массива, представляет собой стопку.

Возьмем для примера стопку тарелок, которую вы используете для приема пищи.

Тарелки стоят друг на друге. В том случае, когда вам нужна тарелка, вы берете ее сверху стопки. Туда же ставите новые или помытые тарелки.

Именно в этом и состоит основная идея стека.



Добавлять элементы можно только «сверху», получать их и удалять можно тоже только через последний элемент. Доступ к середине или началу стека невозможен.

Эта концепция называется **LIFO** (last in, first out или «последним пришел — первым ушел»).

Стандартная реализация в Java так и называется: `Stack`. Используется тем же способом, что и другие коллекции:

```
Stack<Integer> stack = new Stack<>();
```

Методы данной структуры отличаются от стандартных `add`, `get` и `remove`.

В данной коллекции имеется следующий набор методов:



push(E item) — добавление элемента в конец (верх стопки).

peek() — получение последнего элемента.

pop() — получение и удаление последнего элемента.

empty() — проверка на отсутствие элементов.

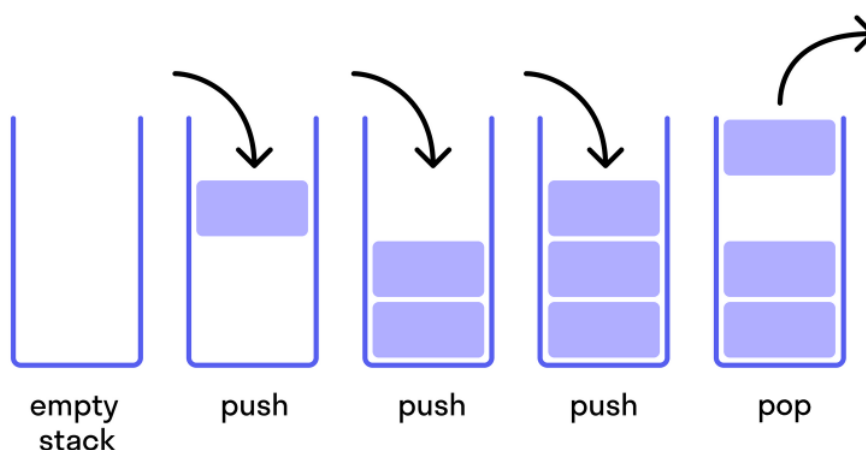
search(E item) — получение номера элемента относительно последнего элемента.

Использование данной коллекции очень узкое, и популярность в связи с этим низкая.

Плюс к этому сама реализация построена на базе старой версии аналога `ArrayList`, класса `Vector`, которая является потокобезопасной и из-за этого далеко не самой быстрой коллекцией, т. к. каждый метод вектора, как и стека, является синхронизированным.

Выглядит стек следующим образом:

Stack



Queue/Deque



Queue, или очередь, является структурой, которая полностью противоположна стеку. Эта структура, как и обычная очередь в реальном мире, позволяет добавить элемент в конец, а получить из начала.

Реализует концепцию **FIFO** (first in, first out или «первым пришел — первым ушел»).

Данная структура имеет как стандартизированный набор методов для стандартных операций, так и специфический.



add/offer(E item) — добавление элемента в конец. Если элемент не удастся добавить, первый вариант выкидывает исключение, второй возвращает false.

peek/element() — получение элемента из начала. Если очередь пуста, первый вариант вернет null, второй выбросит исключение.

poll/remove() — получение и удаление элемента из начала. Если очередь пуста, первый возвращает null, второй — исключение.



Deque, или двусторонняя очередь, позволяет добавлять и получать элементы как из конца, так и из начала.

По аналогии с очередью двусторонняя очередь имеет такой же контракт по названиям методов и возвращаемому результату, но каждый метод продублирован как для головы очереди, так и для хвоста.



addFirst/offerFirst(E item) — добавление в голову очереди.

addLast/offerLast(E item) — добавление в хвост очереди.

peekFirst/getFirst() — получение головы очереди.

peekLast/getLast() — получение хвоста очереди.

removeFirst/pollFirst() — получение и удаление головы очереди.

removeLast/pollLast() — получение и удаление хвоста очереди.

Стандартных реализаций несколько: `ArrayDeque` и уже знакомый по коллекциям `LinkedList`.

```
Queue<Integer> numsArrayQueue = new ArrayDeque<>();
Queue<Integer> numsLinkedList = new LinkedList<>();

Deque<Integer> numsArrayDeque = new ArrayDeque<>();
Deque<Integer> numsLinkedList = new LinkedList<>();
```

Также имеется очередь с приоритетами, которая при добавлении элемента помещает его не в конец, а на позицию в соответствии с его приоритетом. Эта очередь добавит элемент на позицию по приоритету, который указан в компараторе конструктора.

Попробуем добавить числа в случайном порядке, а затем напечатаем их. Числа должны быть выведены в стандартном порядке, т. е. по возрастанию.

```
Queue<Integer> numsPriorityQueue = new PriorityQueue<>(Comparator.comparingInt(e -> e));
numsPriorityQueue.add(3);
numsPriorityQueue.add(1);
numsPriorityQueue.add(2);
while (!numsPriorityQueue.isEmpty()){
    System.out.println(numsPriorityQueue.poll());
}

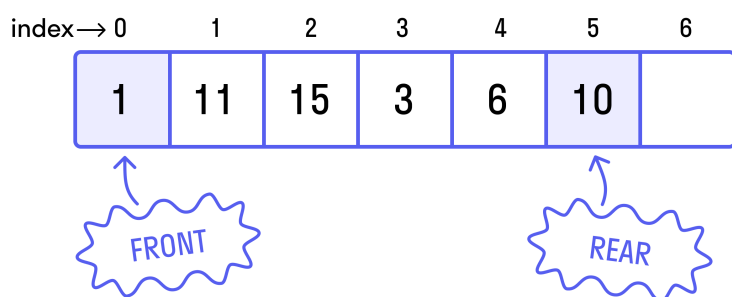
// Результат:
// 1
// 2
// 3
```

Очередь достаточно часто применяется в качестве транспорта для обмена сообщениями между несколькими сервисами, однако в этой ситуации очередь обычно представляет собой отдельную независимую программу. Такие очереди называют брокерами сообщений или MQ (message broker).

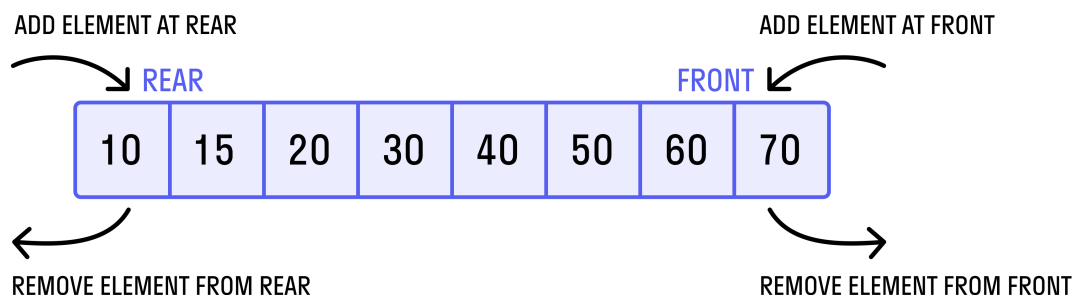
В Java-коде очереди используются, например, для распределения задач между потоками в пуле потоков. Разработчик «накидывает» асинхронные задачи, а потоки по мере освобождения получают из очереди задачу и выполняют ее.

Выглядит очередь схематически так.

Очередь



Двусторонняя очередь



Tree



Деревья представляют собой цепочку из узлов, которые очень похожи на LinkedList, но немного более сложные.

Дерево начинается с узла, который называется **root**. Каждый узел хранит в себе определенное значение (ключ), который и используется для построения дерева, а также ссылки на левый и правый узел.

Рассмотрим в простом виде алгоритм работы самого простого бинарного дерева.

Когда мы создаем дерево, первый элемент становится тем самым корнем. При добавлении каждого следующего элемента он сравнивается с ключом текущего корня. Если ключ нового элемента меньше ключа корневого элемента, то он добавляется влево, иначе — вправо.

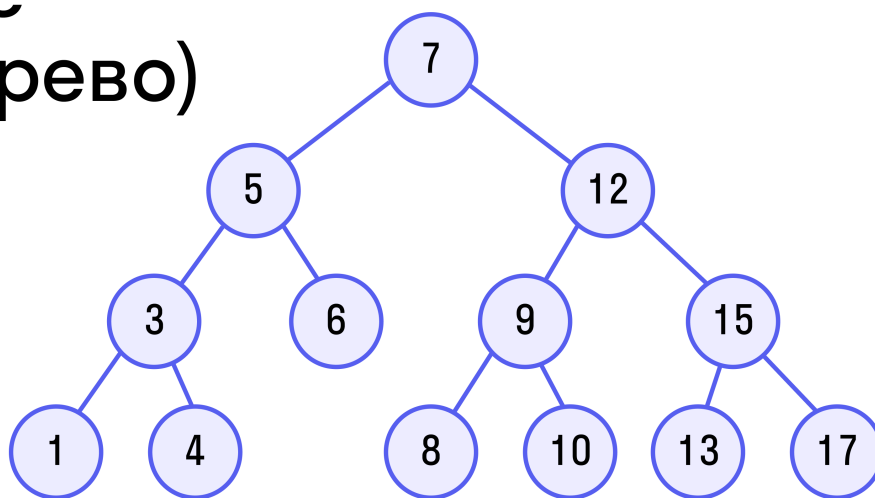
Если слева/справа есть другой узел, то и он сравнивается с ключом нового элемента. Если ключ меньше, то мы снова переходим по ссылке влево, иначе — вправо. И так до тех пор, пока свободная позиция не будет найдена. Когда позиция найдена, узел укладывается в эту позицию.

Стандартными реализациями являются **TreeSet** и **TreeMap**. Это варианты сета и карты, которые поддерживают отсортированный порядок элементов.

```
Set<Integer> nums = new TreeSet<>();  
nums.add(3);  
nums.add(1);  
nums.add(2);  
System.out.println(nums);  
  
// Результат:  
// [1, 2, 3]
```

Если попытаться нарисовать дерево, то выглядело бы оно так.

Tree (дерево)



Graph



Графы похожи на вершины, которые соединены друг с другом.

Обычно построение графов происходит с помощью узлов, которые ссылаются на предыдущий узел, а также через матрицу смежности.

В случае, когда нужно добавить новую вершину (узел), создается вершина, в нее кладется объект, а уже эта новая вершина начинает ссылаться на какую-то из ранее созданных вершин. Это достаточно сложная структура, потому

даже стандартных реализаций в Java-библиотеке нет, т. к. каждый из графов может отличаться по реализации друг от друга.

Используется эта структура, например, для построения маршрутов для курьеров или путешественников, которым нужно пройти через определенное количество точек с минимальными временными затратами.

Графы могут выглядеть как хаотично, так и похоже на дерево.

Типы графов

